
Plateforme agricole intelligente pour les agriculteurs malagasy

VOLY SAINA+ est une application web qui permet aux agriculteurs de Madagascar de consulter des guides de plantation, de louer des machines agricoles et de commander des engrais — le tout depuis une interface simple et adaptée aux conditions rurales. Un espace gestionnaire complet permet de suivre les réservations, les stocks, la facturation et les rapports d'activité.

Sommaire

-  VOLY SAINA+
 -  Sommaire
 - Contexte du projet
 - Technologies utilisées
 - Préparer son environnement
 - Prérequis
 - Configurer application.properties
 - Initialiser la base de données
 - Lancer le projet
 - Architecture du projet
 - Structure de la base de données
 - Utilisateurs
 - Guide de plantation
 - Machines et locations
 - Produits et commandes
 - Facturation
 - Gestion interne
 - Schéma simplifié des relations principales
 - Acteurs et droits
 - Règles de gestion principales
-

Contexte du projet

À Madagascar, une grande partie de la population dépend de l'agriculture, mais beaucoup d'agriculteurs n'ont pas accès à des conseils fiables, ni les moyens d'acheter des machines coûteuses comme un tracteur ou un motoculteur.

VOLY SAINA+ répond à trois besoins concrets :

1. **Informer** — un guide pédagogique sur les cultures (riz, maïs, arachide...) avec les périodes de plantation, les engrais recommandés, les maladies courantes et les outils nécessaires.
2. **Équiper** — un système de location de matériels agricoles (tracteur, motoculteur, pulvérisateur, remorque) sans achat direct.
3. **Approvisionner** — une boutique d'engrais et de produits agricoles (NPK, urée, compost...) avec suivi des stocks et livraison.

Le projet est prévu pour démarrer dans une zone pilote puis s'étendre progressivement vers d'autres régions.

Technologies utilisées

Couche	Technologie
Backend	Java — Spring Boot
Base de données	PostgreSQL
ORM	Spring Data JPA / Hibernate
Build	Maven
Configuration	<code>application.properties</code>
Schéma SQL	Script <code>VolySaina.sql</code> (schéma <code>voly_saina</code>)

Préparer son environnement

Prérequis

Avant de commencer, vérifier que les outils suivants sont installés :

- Java 17+ — <https://adoptium.net>
- Maven 3.8+ — <https://maven.apache.org>
- PostgreSQL 14+ — <https://www.postgresql.org>

Configurer `application.properties`

Le fichier `src/main/resources/application.properties` n'est **pas versionné** (voir `.gitignore` ci-dessous).
Un fichier exemple est fourni : `src/main/resources/example_application.properties`.

Étapes :

1. Copier le fichier exemple :

```
cp src/main/resources/example_application.properties src/main/resources/application.properties
```

2. Ouvrir `application.properties` et renseigner les valeurs selon son environnement :

```
spring.application.name=voly-saina

# Port du serveur (8080 par défaut)
server.port=8080

# Connexion à la base de données PostgreSQL
spring.datasource.url=jdbc:postgresql://localhost:5432/volysaina_db
spring.datasource.username=voly_user
spring.datasource.password=MOT_DE_PASSE_ICI

# Gestion du schéma Hibernate
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
```

⚠ Ne jamais committer `application.properties` avec de vrais identifiants ou mots de passe.

Initialiser la base de données

1. Se connecter à PostgreSQL et créer la base :

```
CREATE DATABASE volysaina_db;
```

2. Exécuter le script SQL fourni pour créer le schéma et les données de départ de la dernière version au format `VolySaina-DD-MM-HH:mm:ss#####.sql` ;

```
psql -U voly_user -d volysaina_db -f database/VolySaina-DD-MM-HH:mm:ss#####.sql
```

Ce script crée le schéma `voly_saina` , toutes les tables, les index, et insère les données initiales (catégories de produits, machines et engrais de base).

Lancer le projet

```
# Cloner le dépôt
git clone https://github.com/votre-utilisateur/voly-saina.git
cd voly-saina

# Compiler et lancer
mvn spring-boot:run
```

L'application sera disponible sur `http://localhost:8080` .

Architecture du projet

Le projet suit une architecture Spring Boot classique en couches :

```

src/
└─ main/
   └─ java/
      └─ com/volysaina/
         ├── controller/    ← Points d'entrée REST (API HTTP)
         ├── service/       ← Logique métier et règles de gestion
         ├── repository/    ← Accès à la base de données (Spring Data JPA)
         ├── model/         ← Entités JPA (tables de la BDD)
         └─ dto/            ← Objets de transfert de données (requêtes / réponses)
      └─ resources/
         ├── application.properties    ← Config locale (non versionnée)
         └─ example_application.properties ← Modèle de config (versionné)

database/
└─ VolySaina.sql    ← Script de création du schéma PostgreSQL

```

Structure de la base de données

Le schéma PostgreSQL s'appelle `voly_saina` et contient les tables suivantes, organisées par domaine fonctionnel.

Utilisateurs

Table	Description
<code>utilisateur</code>	Compte de chaque utilisateur (nom, email, rôle, statut)
<code>profil_utilisateur</code>	Informations complémentaires du profil (âge, genre, localisation, CSP)

Les rôles possibles sont : `client` , `gestionnaire` , `responsable` , `employe` .

Guide de plantation

Table	Description
<code>culture</code>	Catalogue des cultures (riz, maïs, arachide...) avec saison et localisation recommandées
<code>fiche_culture</code>	Fiche détaillée de chaque culture : semences, engrais, arrosage, maladies, conseils

Les fiches doivent être validées (`valide = TRUE`) avant d'être publiées aux clients.

Machines et locations

Table	Description
<code>machine</code>	Machines disponibles à la location avec prix, état et localisation
<code>reservation_machine</code>	Demandes de réservation avec dates, lieu de livraison, prix total et statut
<code>retour_machine</code>	Retour d'une machine après location (état, remarque, pénalité éventuelle)
<code>maintenance_machine</code>	Historique des maintenances planifiées ou réalisées

Les états d'une machine : disponible , louee , maintenance , hors_service . Les statuts d'une réservation : en_attente , validee , refusee , en_cours , terminee , annulee .

Produits et commandes

Table	Description
categorie_produit	Catégories de produits (Engrais, Produit entretien, Produit agricole)
produit	Catalogue des produits avec prix, stock, seuil d'alerte et date d'expiration
mouvement_stock	Historique de chaque entrée, sortie ou correction de stock
commande	Commandes des clients avec adresse de livraison et mode de paiement
ligne_commande	Détail de chaque commande (produit, quantité, prix unitaire, sous-total)

Facturation

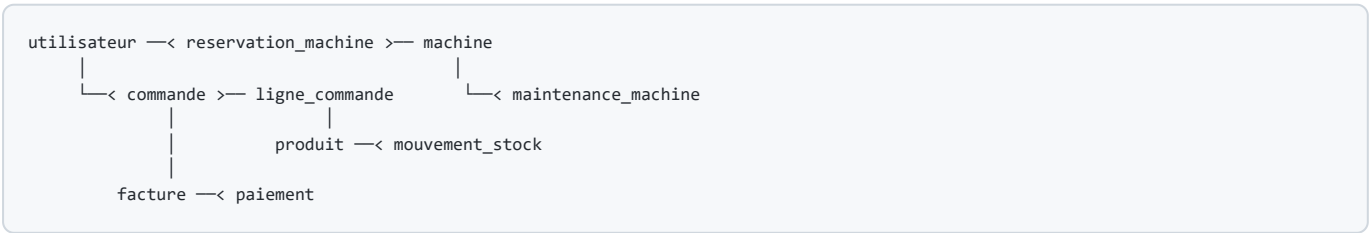
Table	Description
facture	Factures liées à une location ou une commande, avec montant et statut
paiement	Paiements enregistrés sur une facture (montant, date, référence)

Les statuts d'une facture : en_attente , payee , partiellement_payee , en_retard , annulee .

Gestion interne

Table	Description
note_client	Notes internes ajoutées par le gestionnaire sur un client
tache_employe	Tâches assignées aux employés, liées optionnellement à une réservation ou commande
pret_bancaire	Prêts souscrits avec banque, montant, durée et taux
remboursement_pret	Remboursements effectués sur un prêt avec justificatif
rapport	Rapports générés stockés en JSON avec leur période et type

Schéma simplifié des relations principales



Acteurs et droits

Acteur	Accès
Client / Agriculteur	Guide de plantation, location de machine, achat de produits, mes factures, mes statistiques
Gestionnaire	Tableau de bord, machines, réservations, produits, commandes, clients, facturation, employés, rapports
Responsable	Tous les droits du gestionnaire + validation des prix, suivi des KPI, remboursements bancaires
Employé	Tâches assignées : suivi machine, livraison, stock, facturation selon le poste

Règles de gestion principales

- **RG07** — Un client ne peut réserver une machine que si elle est disponible pour la période demandée.
- **RG08** — Deux clients ne peuvent pas réserver le même matériel pour les mêmes dates.
- **RG09** — Le prix de location est calculé automatiquement selon la machine choisie et la durée.
- **RG10** — Une machine en maintenance ou hors service n'apparaît pas comme disponible.
- **RG14** — Un client ne peut pas commander une quantité supérieure au stock disponible.
- **RG15** — Le stock diminue après validation d'une commande et augmente après une entrée de stock.
- **RG16** — Une facture est générée automatiquement après la validation d'une location ou d'une commande.
- **RG20** — Les fiches de culture doivent être validées avant publication.
- **RG21** — Une alerte est déclenchée lorsque le stock d'un produit passe sous son seuil minimum.
- **RG24** — L'interface doit rester légère pour les utilisateurs ayant une connexion faible.

Projet développé pour répondre aux besoins des agriculteurs malagasy dans le cadre d'une initiative d'agriculture intelligente.